Introduction to MLOps and ML System Design
Lab #1
Use case and system design

Assignment overview and motivation

- *Format: Individual, with an online defense during a practical session*
- *Grading: 20 points, one of five mandatory labs worth 20 points each (100 in total)*

This course has one project. You will build it over five labs, each adding one layer of operational maturity to the same system: this lab designs it, Lab 2 makes its data and training automatic and traceable, Lab 3 adds an LLM component with evaluation, Lab 4 deploys and monitors it, and Lab 5 lets an agent use it. The final session demonstration walks through the result. Nothing in the five labs grades how good your model is. Everything in them grades whether your system is reproducible, integrated, observed, and understood by you.

You already know machine learning. What you will develop in this lab is the part that is usually missing when a model meets the real world:

- *Writing down what a system promises before any pipeline exists: who consumes the predictions, how fresh they must be, what happens when they are wrong.*
- *Separating three kinds of success metric (business, model, system) and giving each a number or an honest assumption.*
- *Turning "requirements" into functional and non-functional lists that carry numbers, following an industry design-document template.*
- *Drawing a system context diagram, a data flow diagram and a first deployment diagram that agree with each other and with the text.*
- *Judging whether a dataset can support a system that must keep learning, rather than a one-off notebook.*
- *Starting a repository the way later labs need it: pinned environment, one-command setup, a linter that runs on every push.*
- *Planning the next four labs as milestones of one design, so each later lab revises a document instead of starting over.*

The five labs

One project, five labs. Each lab builds on the artifacts of the one before it, and the final session demonstration walks through the result. All five labs are mandatory and weigh the same: 20 points each, 100 in total.

Lab	Topic	Points
1	Use case and system design	20
2	Data pipelines, versioning, baseline and MLflow	20
3	LLM baseline and retrieval	20
4	Serving on Minikube, monitoring and drift	20
5	Agentic extension	20

A missed lab must be caught up before the next lab that depends on it; if you fall two labs behind, contact the instructor to agree on a reduced scope rather than abandoning the pipeline.

Task

Prerequisites

- *You need a GitHub account, Docker Desktop, `uv` and Python 3.12 or later on your machine, and the lecture of Session 1.*
- *Read Appendix 1 before choosing a use case, and Appendix 3 before drawing anything.*

1. Choose a use case and the data

- *Choose a use case. Appendix 1 lists six problem statements you may take as they are, adapt, or replace with your own. A proposal of your own* **must be approved by the instructor before Session 2**.
- *Choose the data yourself. The choice is yours, and what is graded is your justification. Justify the data in the design document against every item of this checklist, in one or two sentences each:*
  - **A time axis.** *New batches can arrive, so that Lab 2 can ingest "the next month" and Lab 4 can emulate drift. If the data has no timestamps, state how you will manufacture batches and what that costs you.*
  - **Enough history to hold out a period.** *Validation in Lab 2 is by time, not by random split. Say how much history you have and which period you will hold out.*
  - **Labels with a known arrival delay.** *Say when the ground truth for a prediction becomes available (immediately, after a day, after a quarter, never). Lab 4 monitors differently depending on the answer.*
  - **A text surface or a documented corpus.** *Lab 3 adds an LLM component that retrieves from the project's own documents. For a text task that is the data itself; for a numeric task name the corpus you will use (product descriptions, policies, regulations, reports). "None yet, will find one" is not an answer.*
- *If your data fails a checklist item, write the plan that compensates for it. A stated plan scores as complete; a missing item does not.*

2. Write the design document

- *Create `docs/design.md` in your project repository with the sections listed below, in this order and with this numbering. It follows the ml-design-docs template (Appendix 1). Keep the headings and their numbering: every later lab names the sections you must revise by these numbers.*
- *Section by section, this is what "complete" means for a course project:*

- **1 Overview.** *Three to five sentences: problem, solution, desired outcome. This is the anchor of the document and is not revised later.*
- **2 Motivation.** *Why this needs a system and not a notebook: who consumes the predictions, how often, and what happens when they are wrong.*
- **3 Success metrics.** *Three layers, kept apart, each with a target or a stated assumption: a business or decision metric, a model metric, a system metric (freshness, p95 latency, availability or similar). A design that lists only a model metric is incomplete.*
- **4 Requirements and constraints.** *A functional list (inputs, outputs, who calls, what is returned). A non-functional list with* **one row per category***: freshness, latency and throughput, availability and fallback, reproducibility, adaptability (how new data and new models enter), observability without ground truth, cost and resource ceilings.* **Every row carries a number or an explicit assumption***, for example "latency budget unknown until a consumer is defined, assumed 500 ms p95". A missing row is incomplete; a stated assumption is complete. Then in scope and out of scope, explicitly.*
- **5.1 Problem statement.** *How you frame the problem (classification, regression, forecasting, ranking, anomaly detection) and one rejected framing with the reason.*
- **5.2 Data.** *Sources, arrival pattern, labels and their delay, what is available at serving time versus training time, and the checklist justification from step 1.*
- **5.3 Techniques.** *The baseline first (a rule or a trivial model), then the candidate. A feature list with, for every feature, where its value will come from at serving time.*
- **5.4 Experimentation and validation.** *The split strategy, justified by the time axis. The offline metric. How a new model will be compared with the current one.*
- **5.5 Human in the loop.** *Who reviews a drift alert, who approves a model promotion, what an override looks like.*
- **6.1 High-level design.** *The three diagrams of step 3.*
- **6.2 Infra.** *The local stack you will run and, per component, the managed cloud service you would use instead. Use the component names and the cloud columns of the reference table in Appendix 3; naming them is required, running on them is not.*
- **6.3 Performance.** *Throughput and latency budgets, and the batch-versus-online decision with a reason.*
- **6.4 Security and 6.5 Data privacy.** *Where secrets live, who may call the service, what personal data the dataset contains and how long you keep it. Short, but not empty.*
- **6.6 Monitoring and alarms.** *What you will log, which metrics you will compute, which threshold will fire, and who receives the alert. First draft; Lab 4 makes it real.*
- **6.7 Cost.** *An estimated monthly cost of the cloud column, even though you will run locally. Use the providers' calculators and state your assumptions.*
- **6.8 Integration points.** *The upstream owner of the data, the downstream consumer of the predictions, and the contract between you and each.*
- **6.9 Risks and uncertainties.** *Named risks, one mitigation each.*
- **7.1 Alternatives.** *At least two choices you rejected (a framing, a tool, a data source, an architecture) with pros, cons and the reason.*
- **7.2 and 7.3.** *Placeholders; Lab 2 and Lab 4 fill them.*
- **7.4 Milestones.** *One sentence per later lab on what it will add to this system.*

- *Where you cannot answer yet, write the assumption you proceed with and mark it as an assumption. Never leave a section empty.*

3. Draw the three diagrams

- **System context.** *Your system as one box. Around it, every external party: each data source, the label source, each consumer of predictions, and the person or channel that receives alerts. Label the arrows with what flows.*
- **Data flow.** *Draw it on the seven layers of a production ML system: data ingestion, preprocessing and transformation, a feature store with an offline and an online side, the training pipeline, the model registry and deployment, inference (real-time, batch or both), and the monitoring and retraining loop that feeds back into training. This is the layering used in ML system design interviews (see Appendix 1); name your own component inside each layer, and write "not needed" with a reason for a layer your system skips. On it, mark three things explicitly: where a data version is taken, where a quality check runs, and where the training path and the serving path diverge (in this layering, at the offline and online sides of the feature store). These marks are the point of the diagram; boxes and arrows without them are a data-science sketch, not a system design.*
- **Deployment, first draft.** *The components you will run locally, each with its cloud equivalent named next to it. Lab 4 replaces this diagram with the real cluster topology, so you do not need Kubernetes vocabulary yet.*
- *The three diagrams must name the same components as the text. A component in a diagram but not in section 6.2, or the reverse, scores as inconsistent.*
- *Appendix 3 has a worked example of each diagram for a generic system, and a blank template of each. Redraw the template in any diagram tool.* **A worked example submitted with no component renamed to your own system scores zero for the diagrams row.**

4. Set up the repository

- *A repository on GitHub, public or shared with the instructor, with:*
  - `README.md` *: the problem in five sentences and the Lab 1 rows of the maturity scorecard (Appendix 3).*
  - `docs/design.md` *: the design document with the diagrams embedded.*
  - *A pinned Python environment:* `pyproject.toml` *plus* `uv.lock` *(or an equivalent lock file).*
  - *A* `Makefile` *with at least* `setup` *(installs the environment) and* `lint` *(runs the linter).*
  - *A pre-commit configuration that runs the same linter.*
  - *A CI workflow that runs* `make lint` *on every push. Lint only; a minimal workflow is in Notes.*
- *The last CI run on the default branch is green at submission time. A reviewer on a clean clone runs* `make setup` *and* `make lint` *and both succeed.*

5. Submit

- *Tag the commit you submit as* `lab-1` *and send the repository link. The defense walks through the design document and the diagrams, not the code, because there is no code yet.*

Notes

- **The choice of data and model is yours and is graded on justification.** *Nothing in the grading rewards a sophisticated model, in this lab or in the four that follow. A system that nobody can run scores lower than a simple model that a reviewer can rebuild from the repository.*
- **Assumptions beat empty sections.** *Every section of the template can be answered with "Assumption: ..." when you do not know. Unknowns you have named are design decisions; unknowns you have hidden are bugs.*

- **The worked example is an example.** *Appendix 3's diagrams use generic names on purpose. Redraw with your component names.*
- **CI beyond lint is deferred.** *Lab 2 adds tests to the pipeline. For now the workflow only has to prove that the repository is in a state a machine can check. A minimal GitHub Actions workflow:*

```
name: lint
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
      - run: make setup
      - run: make lint
```

- *A* `Makefile` *that satisfies step 4:* `setup: ; uv sync` *and* `lint: ; uv run ruff check .` *on separate lines, with a tab before each command.*

Appendix 1 (Recommended sources)

The template and the report you are imitating

- The ML design-doc template this lab follows, section by section: https://github.com/eugeneyan/ml-design-docs
- A student report that reads like a design document and then reports what was built, from Stanford CS 329S: https://stanford-cs329s.github.io/reports/ml-production-system-for-covid-detection/ . Note how it names the cloud services it used, what it changed after launch and why.
- The whole CS 329S syllabus, the course this one draws on most: https://stanford-cs329s.github.io/syllabus.html
- The seven-layer architecture the data flow diagram follows (ingestion, preprocessing, feature store, training, registry and deployment, inference, monitoring and retraining), from the System Design Handbook's guide to the Google ML system design interview: https://www.systemdesignhandbook.com/guides/google-ml-system-design-interview/

Use-case variants (problem statements only; data and model are your choice)

- Dynamic news classification. Classify news articles into categories, adapting to topics and language that change over time. Pipeline: periodic collection, preprocessing, training, monitoring for degradation, trigger-based retraining.
- Seasonal sales forecasting. Forecast product sales in a retail setting, adjusting for seasonality and trend. Pipeline: time-series features, training, periodic re-evaluation as each new season arrives, model update.
- Automated social media moderation. Detect harmful content in posts, adapting to new forms of harmful content and usage patterns. Pipeline: streaming or batched ingestion, classification, monitoring for new forms, periodic retraining.
- Dynamic pricing for online retail. Adjust prices from demand, competitor prices and inventory, with continuous adaptation. Pipeline: regular data updates, price model training, monitoring of market conditions, retraining on shifts.
- Customer support query classification. Route support queries to the right department, adapting to new products and new query types. Pipeline: ingestion, classification, routine accuracy evaluation, retraining on new data.
- Environmental monitoring for pollution control. Predict pollutant levels in urban areas, adjusting to new regulations and infrastructure. Pipeline: sensor data ingestion, forecasting, performance monitoring, model updates as data evolves.

Documentation of the services you will name

- The reference stack and its cloud equivalents, with a documentation link per service, is the table in Appendix 3. Each row's AWS, GCP and Azure entries link to the official documentation; read the page of each service you name in section 6.2.
- Pricing calculators for section 6.7: https://calculator.aws/ , https://cloud.google.com/products/calculator , https://azure.microsoft.com/pricing/calculator/

Course reading list (from the syllabus)

- Machine Learning Engineering for Production (MLOps) specialization: https://www.coursera.org/specializations/machine-learning-engineering-for-production-mlops
- Weights and Biases courses, including data validation for machine learning: https://wandb.ai/site/courses/

- MLflow: https://github.com/mlflow/mlflow
- Minikube: https://minikube.sigs.k8s.io/docs/start/
- Designing Machine Learning Systems, Chip Huyen (O'Reilly, 2022): https://huyenchip.com/books/

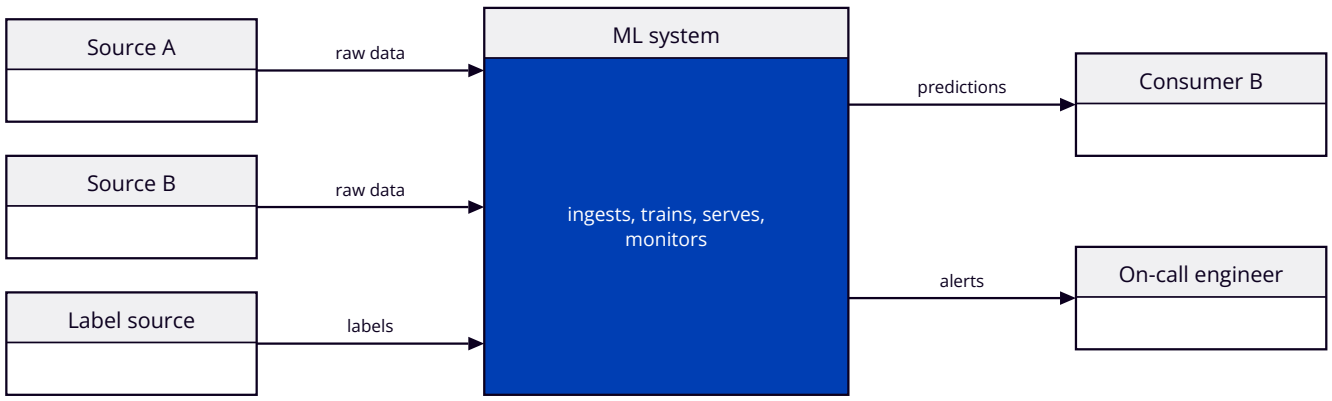
Appendix 2 (Theoretical questions)

Questions marked with an asterisk can only be answered with your own design document open.

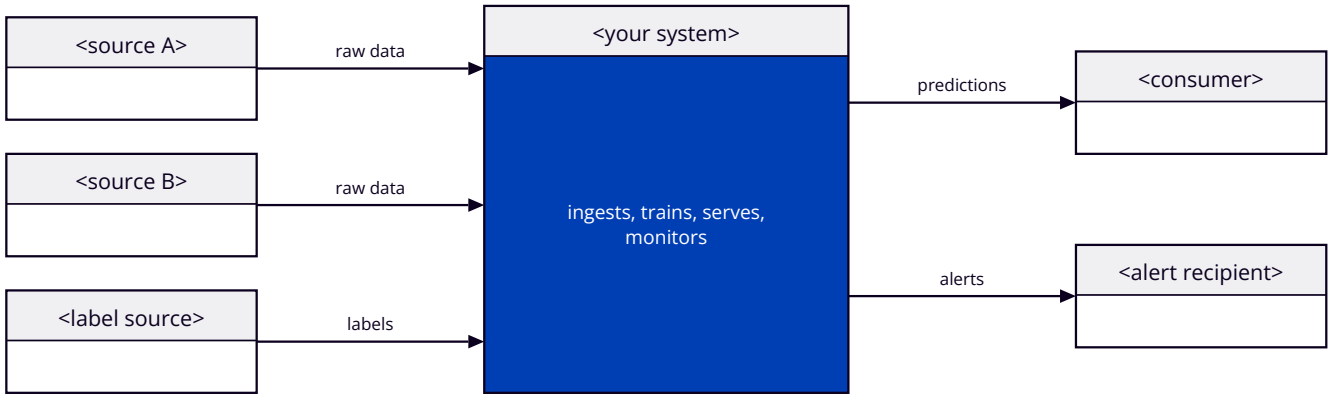
- What distinguishes a production ML system from a research system? Name three requirements the first has that the second does not, and say where each appears in your design.*
- The syllabus names reliability, scalability and adaptability as the requirements of an ML system. Define each in one sentence, and give the non-functional requirement in your document that covers it.*
- Which two of your non-functional requirements conflict with each other, and which one wins? Why?*
- Describe CRISP-DM. Which of its phases does this lab cover, and which phases do Labs 2 to 4 cover?
- What is an optimization metric versus a satisficing metric? Which of your three success metrics is which?*
- Your business metric improves while your model metric stays flat. Give two explanations, and say which one your monitoring would distinguish.*
- Your model metric improves offline and the business metric gets worse after deployment. What in your design could have caused it, and what in section 5.4 was supposed to catch it?*
- Why is a random train-test split wrong for a system that receives data over time? What split does your document use, and what leakage does it prevent?*
- Labels for a prediction arrive two weeks after the prediction. What can you monitor in those two weeks, and what can you not? Which of your requirements rows states this?*
- Your pipeline succeeded for three days and the output table is stale, because the job read the same partition each time. What alert would have caught it, and does section 6.6 of your document contain it?*
- A consumer of your predictions asks for results that are both complete and instant. Explain why they cannot have both, and what your freshness requirement promises instead.*
- What does the model lifecycle look like from the first notebook to the retired model? Which artifact of your repository represents each stage?
- Compare CSV, Parquet and a database table as formats for the raw layer of your system. Which did you choose for raw, clean and features, and why?*
- What does Git LFS do that Git does not, and what does DVC do that Git LFS does not? Which of your data would go to each, and what stays in Git?
- Your data source changes its schema without notice. Where in your data flow diagram is that detected, and what happens downstream of that point?*
- Name the components of your deployment diagram and, for each, the managed cloud service you named. Pick one and say what you would lose and gain by running the managed version.*
- Your monthly cost estimate in section 6.7 turns out to be ten times too low. Which assumption was most likely wrong, and which requirement would you relax first?*
- What is training-serving skew? Point to the place in your data flow diagram where it could enter your system.*
- Why does a feature need a declared serving-time source in section 5.3? Give one feature from your design whose serving-time source is not the same as its training-time source.*
- In which situations is a rule-based baseline the right production model? Does your section 5.3 baseline qualify, and how would you know?*

Appendix 3 (System design diagrams)

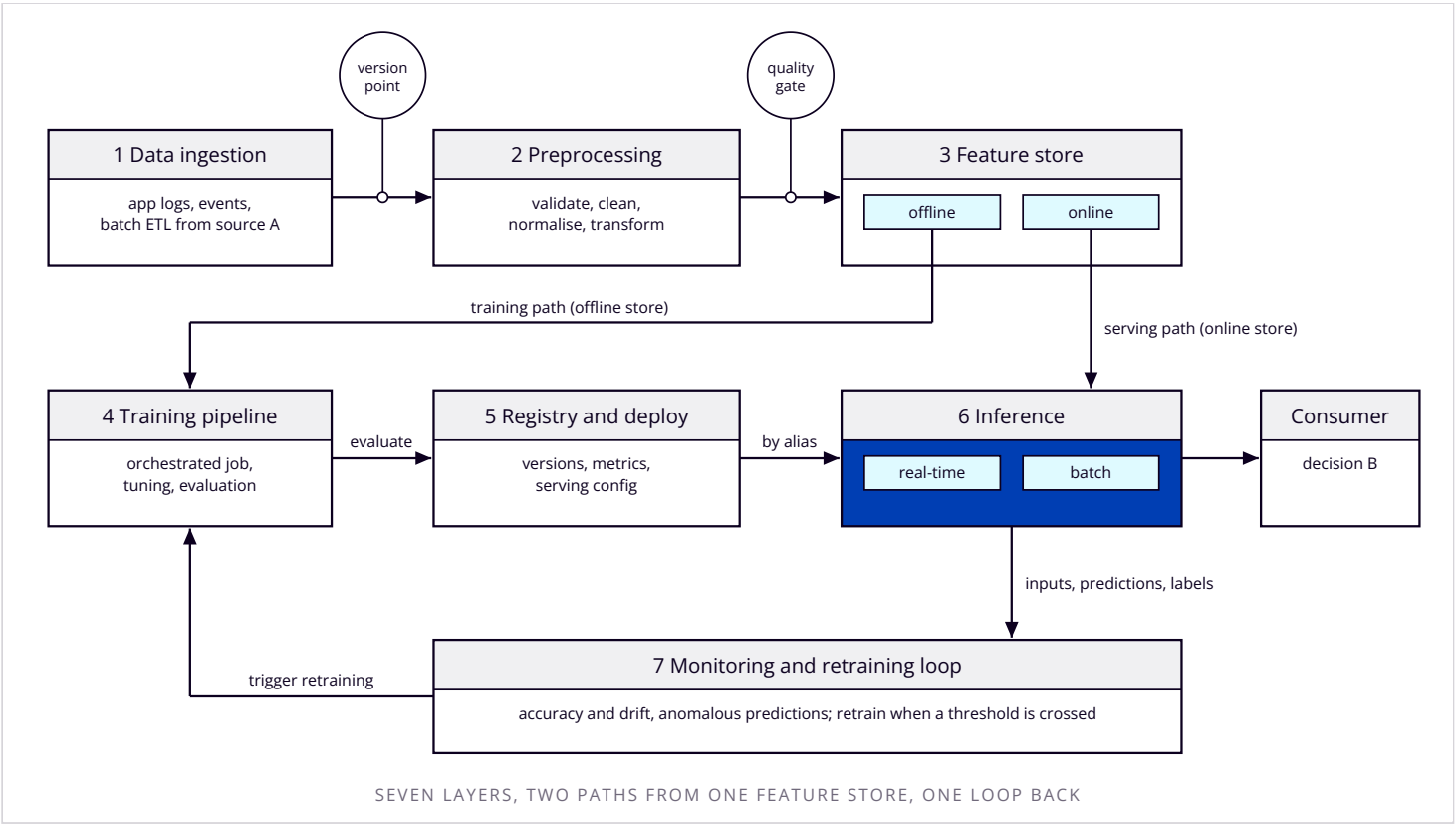
Worked examples use generic names on purpose. Redraw each diagram with your own component names, in any diagram tool, starting from the blank template shown here.



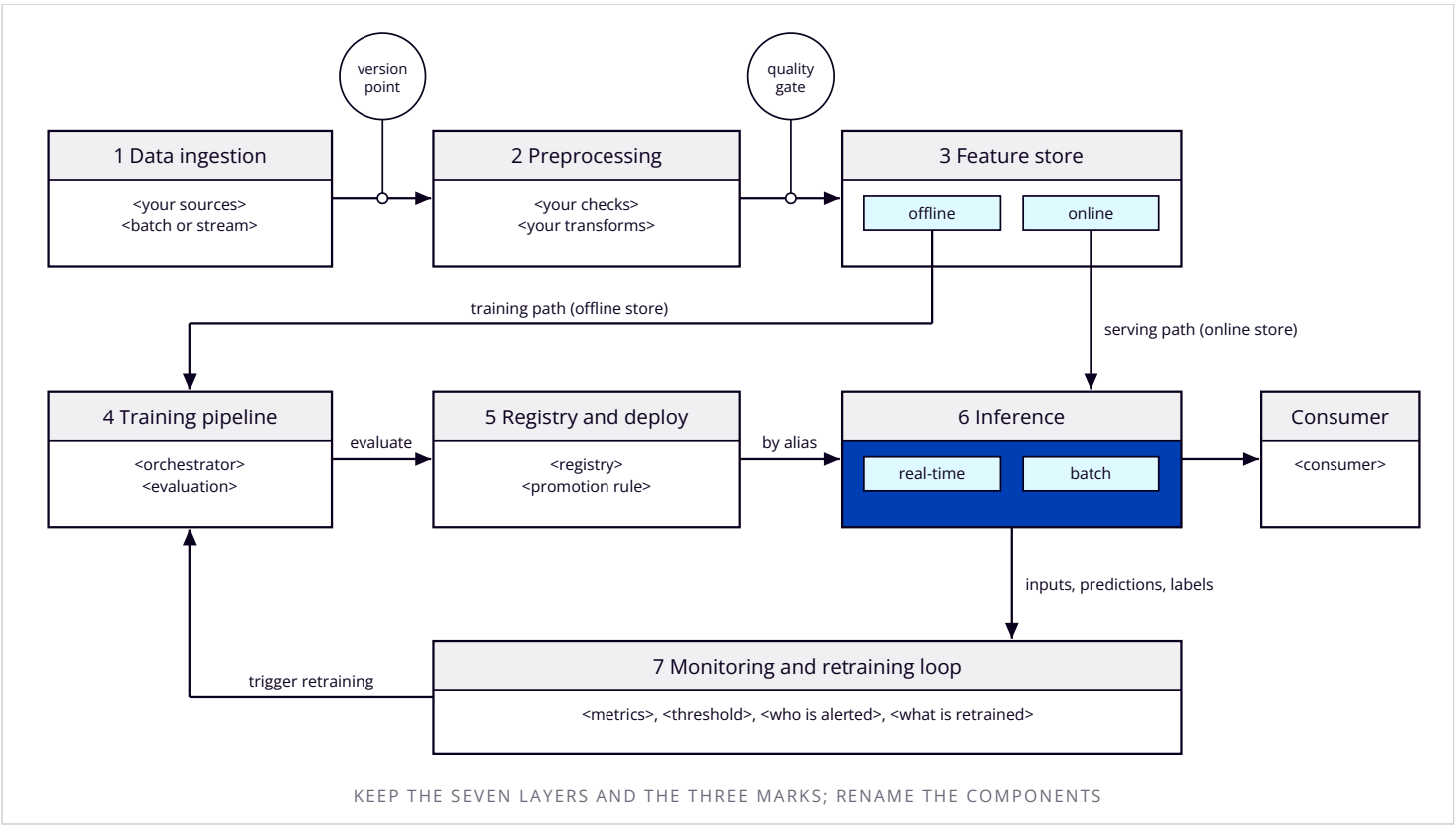
Worked example: system context. The system as one box, every external party around it, arrows labelled with what flows.



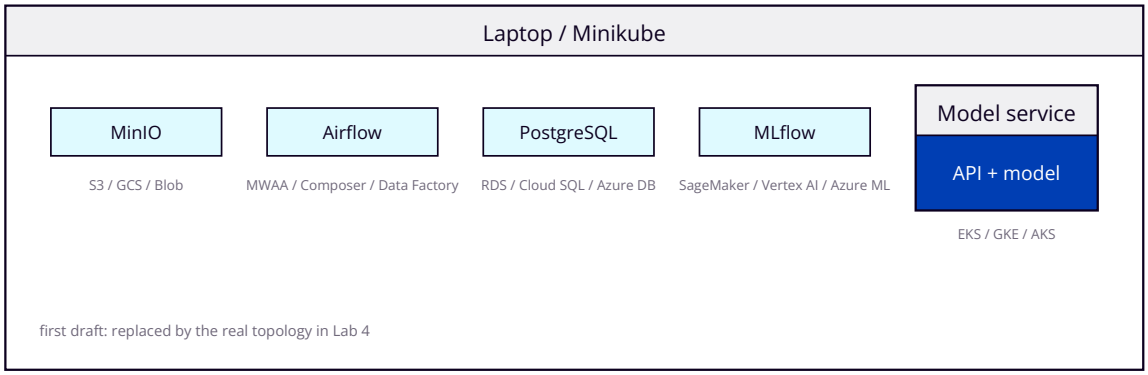
Template: system context. Replace the placeholder labels with your sources, consumers and alert recipient.



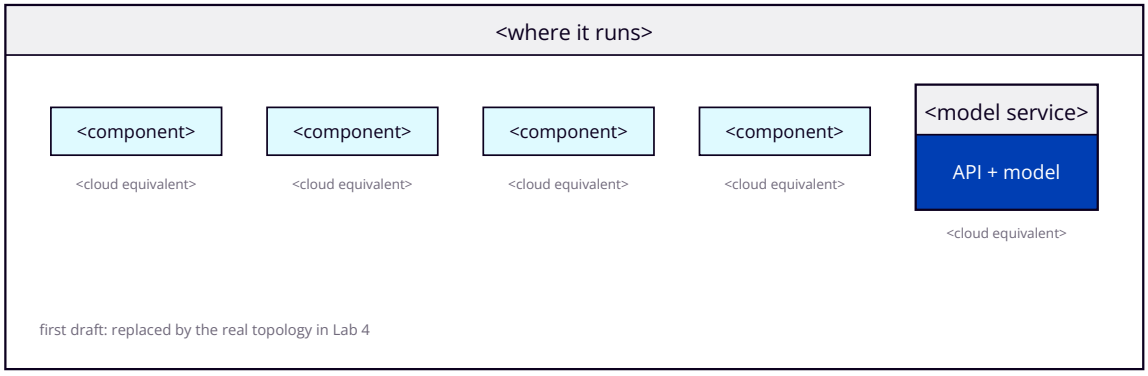
Worked example: data flow on the seven layers. The version point and the quality gate sit before the feature store; the offline side feeds training, the online side feeds inference; monitoring loops back into training.



Template: data flow. Keep the seven layers and the three marks; replace every placeholder with your component.



Worked example: deployment, first draft. Local components with the managed cloud service next to each.



Template: deployment, first draft. Lab 4 replaces this diagram with the real cluster topology.

Reference stack and cloud equivalents

The labs are written against this local stack. Every component is named here once, and the labs use these names. The right-hand columns give the managed service you would use for the same job on each cloud; naming them in your design document is required, running on them is not.

Component	***Local reference (pinned)***	***AWS***	***GCP***	***Azure***
Object storage, data lake	*MinIO-compatible* `pgsty/silo:RELEASE.2026-09-16T00-00-00Z` *(see note below)*	*Amazon S3*	*Cloud Storage*	*Blob Storage*

Component	Local reference (pinned)	AWS	GCP	Azure
Data versioning remote	DVC `3.67.1` with dvc-s3 `3.3.0`, remote on MinIO (DVC remotes)	S3 remote	GCS remote	Azure Blob remote
Warehouse, serving database	PostgreSQL `16`	Amazon RDS	Cloud SQL	Azure Database for PostgreSQL
Orchestrator	Apache Airflow `3.3.1`, LocalExecutor	Amazon MWAA	Cloud Composer	Azure Data Factory
Managed ML pipelines (lecture only)	Airflow DAG	SageMaker Pipelines	Vertex AI Pipelines	Databricks workflows
Experiment tracking and model registry	MLflow `3.16.1`, PostgreSQL backend, MinIO artifacts	SageMaker with MLflow	Vertex AI Experiments and Model Registry	Azure ML with MLflow
Feature store	Feast `0.66.0`	SageMaker Feature Store	Vertex AI Feature Store	Azure ML managed feature store
Container registry	Minikube's built-in registry or Docker Hub	Amazon ECR	Artifact Registry	Azure Container Registry
Kubernetes cluster	Minikube `1.33.1` (Kubernetes `1.30.0`, docker driver, 4 GB)	Amazon EKS	GKE	AKS
Model and data monitoring	Evidently	SageMaker Model Monitor	Vertex AI Model Monitoring	Azure ML model monitoring
Metrics store and dashboards	Prometheus, Grafana	Amazon CloudWatch	Cloud Monitoring	Azure Monitor
Vector store	pgvector `0.8.7` (image `pgvector/pgvector:pg16`) or Qdrant `v1.16.3`	OpenSearch Serverless vector search	Vertex AI Vector Search	Azure AI Search
LLM inference	vLLM `0.30.0` (OpenAI-compatible server; NVIDIA GPU, or CPU in Docker from a locally built image on a Mac) or Unsloth `2026.10.1` (NVIDIA GPU); or an API	Amazon Bedrock	Vertex AI generative AI	Azure OpenAI

The official MinIO images (`quay.io/minio/minio`, `minio/minio`) can no longer be pulled anonymously as of October 2026. `pgsty/silo` is a maintained build of the same server with the same API, ports and environment variables (`MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`); every "MinIO" in these labs means an S3-

compatible store started from that image or any equivalent. Create buckets with `amazon/aws-cli` against the S3 endpoint if the `mc` client image is unavailable.

For projects that keep a streaming path: Apache Spark `4.0.4` and Apache Kafka `4.3.1`.

The maturity scorecard

Your repository README carries one table that grows by a few rows per lab. Each row names a property of your system, the evidence that shows it, and the lab that added it. The final session demonstration walks through this table, row by row, and every row has to point at something a reviewer can reproduce from the repository. Start it in Lab 1 with the first two rows.

Property	***Evidence a reviewer checks***	***Added in***
Repository reproducible from a clean clone	*CI is green; `make setup` and `make lint` succeed on a fresh clone*	*Lab 1*
Design document exists and is current	*`docs/design.md` follows the template, every section filled or marked as an assumption*	*Lab 1*
Data versioned	*the documented pull command retrieves the raw data on a clean clone*	*Lab 2*
Pipeline idempotent	*the same interval run twice gives identical row counts, both pasted in the README*	*Lab 2*
Quality gate present	*one bad batch fails the run on purpose, shown*	*Lab 2*
Experiments tracked	*every training run in the tracking server has parameters, metrics, data version and commit*	*Lab 2*
Model registered with lineage	*a registered version tagged with data version and commit, reproducible by one command*	*Lab 2*
LLM component evaluated	*an evaluation set with retrieval and answer metrics logged as runs*	*Lab 3*
Prompts versioned	*prompt files whose hash appears on every LLM run*	*Lab 3*
Cost and latency recorded	*tokens or compute, cost, p50 and p95 per run*	*Lab 3*
Services containerized	*two images built from the repository, tagged with the commit*	*Lab 4*
Services deployed with health contracts	*both services ready on Minikube, probes fail when the model is missing*	*Lab 4*
Monitoring scheduled	*reports on a schedule, a dashboard with at least two windows*	*Lab 4*
Drift signal reaches a human	*an injected shift crosses the threshold and the alert is received*	*Lab 4*
One command brings the system up	*the documented command starts everything from a clean clone and answers a smoke request*	*Lab 4*
Agent traced and evaluated	*persisted traces and a ten-task evaluation run twice*	*Lab 5*